

SAP Credential Store Client

Module `srv/credentialStore/credentialStore.ts` — reads secrets from the SAP BTP Credential Store over mutual TLS, decrypts each JWE-encrypted response, and returns the plain value.

CollabSphere — SAP CAP App | Instance `Credential_Store` | Namespace `Collabsphere`

1. What this module does

The app needs secrets (AEM broker host, username, password, VPN, queue) without storing them in source, in `.env`, or in the Cloud Foundry environment. This module is a small, self-contained client for the SAP Credential Store REST API. It has no CAP dependency and can be reused for any secret.

Export	Returns	Use
<code>readCredential(name, namespace, kind?)</code>	the full decrypted credential object	when you need <code>id</code> , <code>username</code> , timestamps, etc.
<code>readCredentialValue(name, namespace, kind?)</code>	just the secret string	the common case

`kind` is `"password"` (default) or `"key"`; it must match how the credential was created in the cockpit, because it selects the API path.

2. The service binding

`xsenv.getServices()` reads the bound instance's `credentials` object. On Cloud Foundry it comes from `VCAP_SERVICES.credstore[0]`; locally `cds watch` injects the same shape from the `cred_key` service key. Fields this module uses:

Field	Purpose
<code>url</code>	REST base — <code>https://credstore.mesh.cf.us10.hana.ondemand.com/api/v1/credentials</code>
<code>username</code>	<code>Authorization: Basic base64(username + ":")</code> — identifies the calling binding (password is empty)
<code>certificate</code> + <code>key</code>	client certificate + private key presented in the TLS handshake (mutual TLS)
<code>encryption.client_private_key</code>	RSA private key (base64 DER) that decrypts every response body
<code>encryption.server_public_key</code>	encrypts request bodies — only needed for writes, not used here

3. Two security layers

- **Transport — mutual TLS.** The instance's `authentication.type` is `mtls`. During the handshake the store demands a client certificate and checks it was issued for this instance. A wrong/absent cert fails as `401`.
- **Payload — JWE.** The instance has `encryption.payload: enabled`, so each response body is a JWE compact token encrypted with the public half of `encryption.client_private_key`. Only this process can decrypt it — even a stolen TLS cert does not reveal values.

4. Full source — credentialStore.ts

```
1  import https from "node:https";
2  import fetch from "node-fetch";
3  import jose from "node-jose";
4  import * as xenv from "@sap/xsenv";
5
6  interface CredstoreBinding {
7    url: string;
8    username: string;
9    certificate: string;
10   key: string;
11   encryption: {
12     client_private_key: string;
13     server_public_key: string;
14   };
15 }
16
17 type CredentialKind = "password" | "key";
18
19 interface Credential {
20   id: string;
21   name: string;
22   value: string;
23   username?: string;
24   type: CredentialKind | "keyring";
25   [key: string]: unknown;
26 }
27
28 let cachedBinding: CredstoreBinding | undefined;
29
30 // Resolves the bound `credstore` service instance from VCAP_SERVICES (CF) or the
31 // local default-env file loaded by xenv.loadEnv() in collabSphereModel.ts.
32 function getBinding(): CredstoreBinding {
33   if (cachedBinding) return cachedBinding;
34   const { credstore } = xenv.getServices({
35     credstore: { label: "credstore" },
36   }) as { credstore: CredstoreBinding };
37   cachedBinding = credstore;
38   return credstore;
39 }
40
41 // Wraps a base64-encoded DER key/cert body into PEM so node-jose can import it.
42 function toPem(base64Der: string, label: string): string {
43   const body = base64Der.replace(/(.{64})/g, "$1\n").trimEnd();
44   return `-----BEGIN ${label}-----\n${body}\n-----END ${label}-----\n`;
45 }
46
47 // The Credential Store instance has payload encryption enabled, so every response
48 // is a JWE compact string encrypted for the binding's client key.
49 async function decryptPayload(
50   jwe: string,
51   binding: CredstoreBinding,
52 ): Promise<Credential> {
53   const privateKey = toPem(binding.encryption.client_private_key, "PRIVATE KEY");
54   const key = await jose.JWK.asKey(privateKey, "pem");
55   const { plaintext } = await jose.JWE.createDecrypt(key).decrypt(jwe);
56   return JSON.parse(plaintext.toString("utf8")) as Credential;
57 }
58
59 // Reads a single credential from the given namespace. `kind` selects the API
60 // path segment (.../credentials/password or .../credentials/key) and must match
61 // how the credential was created in the cockpit. Throws on any failure - there
62 // is no fallback, so a missing binding or a failed read stops startup.
63 export async function readCredential(
```

```

64     name: string,
65     namespace: string,
66     kind: CredentialKind = "password",
67   ): Promise<Credential> {
68     const binding = getBinding();
69     const agent = new https.Agent({
70       cert: binding.certificate,
71       key: binding.key,
72     });
73     // mTLS binding: username in the Authorization header, empty password.
74     const auth = Buffer.from(`${binding.username}:`).toString("base64");
75
76     const response = await fetch(
77       `${binding.url}/${kind}?name=${encodeURIComponent(name)}`,
78       {
79         method: "GET",
80         agent,
81         headers: {
82           Authorization: `Basic ${auth}`,
83           "sapcp-credstore-namespace": namespace,
84         },
85       },
86     );
87
88     if (!response.ok) {
89       throw new Error(
90         `Credential Store read failed for "${name}" (${kind}) in namespace ` +
91         `"${namespace}": ${response.status} ${response.statusText}`,
92       );
93     }
94
95     return decryptPayload((await response.text()).trim(), binding);
96   }
97
98   // Returns just the secret value. "key" credentials store the value base64-encoded;
99   // "password" credentials store it verbatim.
100  export async function readCredentialValue(
101    name: string,
102    namespace: string,
103    kind: CredentialKind = "password",
104  ): Promise<string> {
105    const credential = await readCredential(name, namespace, kind);
106    return credential.type === "key"
107      ? Buffer.from(credential.value, "base64").toString("utf8")
108      : credential.value;
109  }

```

5. Code walkthrough

Imports (lines 1–4)

`node:https` builds an agent carrying the client cert. `node-fetch` is used instead of the global `fetch` because only it accepts a custom `agent` (needed for mTLS). `node-jose` decrypts the JWE — imported as a *default* import because it is CommonJS; `import * as` would hide its exports under `.default` under tsx/esbuild. `@sap/xsenv` reads the binding.

Types (lines 6–26)

`CredstoreBinding` is the subset of binding fields used here. `CredentialKind` restricts `kind` to the two readable API paths. `Credential` is the decrypted JSON; `[key: string]: unknown` keeps extra fields (timestamps, status) without typing them all.

getBinding() (lines 28–39)

`xsekv.getServices({ credstore: { label: "credstore" } })` finds the bound service whose offering label is `credstore` and returns its credentials. The result is memoised in `cachedBinding` so the lookup runs once per process, not once per read.

toPem() (lines 42–45)

`encryption.client_private_key` arrives as raw base64 DER with no header lines. `node-jose` needs PEM, so this re-wraps it at 64 characters per line between `-----BEGIN PRIVATE KEY-----` / `-----END PRIVATE KEY-----`.

decryptPayload() (lines 49–57)

1. `toPem( ... )` → PEM string for the client private key.
2. `jose.JWK.asKey(privateKey, "pem")` imports it as a key object.
3. `jose.JWE.createDecrypt(key).decrypt(jwe)` decrypts the compact token (RSA-OAEP-256 + A256GCM, 3072-bit) and yields `{ plaintext: Buffer }`.
4. `JSON.parse(plaintext.toString("utf8"))` → the `Credential`.

readCredential() (lines 63–96)

```
GET {url}/{kind}?name={encodeURIComponent(name)}
Authorization: Basic base64("{username}:") // trailing colon = empty password
sapcp-credstore-namespace: {namespace} // case-sensitive
```

- `new https.Agent({ cert, key })` — the mTLS material; passed to fetch as `agent`.
- `{kind}` — the credential *type* is a path segment (`/key` or `/password`). The bare collection URL returns `400`.
- `!response.ok` → **throw**. No retry, no fallback — the error propagates out of `init()` and the server refuses to start.
- `decryptPayload((await response.text()).trim(), binding)` — the body is a string token (not JSON), trimmed of a trailing newline.

readCredentialValue() (lines 100–109)

Calls `readCredential`, then: for `type` `≡ "key"` the value is base64 → `Buffer.from(value, "base64").toString("utf8")`; for `"password"` the value is returned verbatim.

6. One read, end to end

```
readCredentialValue("AEM_HOST", "Collabsphere", "key")
→ readCredential(...)
  getBinding()                cached VCAP_SERVICES.credstore credentials
  new https.Agent({cert,key})  mTLS client material
  auth = base64("username:")
  fetch GET ... /credentials/key?name=AEM_HOST
    TLS handshake: server validates our client cert
    200 + JWE compact string
  response.ok? no → throw (startup fails)
  decryptPayload(jwe, binding)
    toPem(client_private_key) → jose.JWK.asKey
    jose.JWE.createDecrypt(key).decrypt(jwe) → plaintext Buffer
    JSON.parse → { name:"AEM_HOST", value:"<base64>", type:"key" }
→ type==="key" → Buffer.from(value,"base64").toString("utf8")
"wss://mr-connection-...messaging.solace.cloud:443"
```

7. Usage

```
import { readCredentialValue } from "../credentialStore/credentialStore.js";

const host = await readCredentialValue("AEM_HOST", "Collabsphere", "key");
const smtp = await readCredentialValue("SMTP_APP_PASSWORD", "Collabsphere"); // default "password"
```

8. Errors seen while building this

Symptom	Cause	Fix
400 Bad Request	called the bare <code>/api/v1/credentials</code>	append the type: <code>/key</code> or <code>/password</code>
Cannot read properties of undefined (reading 'asKey')	<code>import * as jose</code> from a CJS package under tsx/esbuild nests exports on <code>.default</code>	<code>import jose</code> from "node-jose"
Service instance 'credstore' not found	wrong name / reversed <code>cf bind-service</code> args	instance is <code>Credential_Store</code> ; <code>cf bind-service <app> <instance></code>

Source: [srv/credentialStore/CredentialStore_Explained.html](#) . Regenerate the PDF with headless Chrome `--print-to-pdf` .